

PIO Manage Desk Portal

Project dossier, 28 September 2026

Live at piomanage.com

Integrations: Datto Autotask and ConnectWise Manage

Status: in production, early adoption

All figures read from the live system on 28 September 2026

Contents

Section	Page topic
1	Executive summary, status at a glance, live production figures
2	What the product does, by audience
3	Architecture, technology and environments
4	PSA integration, capability matrix and the sync engine
5	Reporting, analytics and email delivery
6	Security and data protection
7	Operations: deployment, backups, monitoring
8	Quality assurance
9	What is outstanding, split into owner actions and engineering gaps
10	Recommended enhancements, in three tiers
11	Risk register
A	Screens and routes
B	API surface
C	Permissions and roles
D	Background processing
E	Where everything lives
F	Delivery history and decisions
G	Infrastructure
H	Continuous integration
I	Test inventory
J	Data model and change history

How to read this document

This is a written record of one product: what exists today, what is unfinished, and what should be built next. It is meant to stand on its own for a reader who has never seen the code or the running system, including an assistant asked to advise on it.

Three conventions are worth knowing before starting.

- **Every figure was read, not estimated.** Counts came from the live production database and the repository on 28 September 2026. Where something is unproven, the document says so rather than rounding up.
- **"Built but not in use" is a category of its own.** A September feature audit found 29 features that are implemented and hold no data in production. They are called out as such, because the cheapest improvements available are switching them on rather than writing anything new.
- **Recommendations carry effort and impact.** Section 10 is ordered by return, and the last part of

it names the three things worth doing first if nothing else is done.

1. Executive summary

PIO Manage (the Desk Portal) is a multi-tenant, self-hosted SaaS ticket portal that puts one modern experience in front of the service-desk platforms an MSP already runs. It is live at **piomanage.com** and currently federates two providers: **Datto Autotask** (production tenant) and **ConnectWise Manage** (a staging tenant). Each PSA stays the system of record. The portal synchronizes, normalizes, and reports on top of it, and writes back tickets, notes, statuses and time entries through the same connectors.

The product serves four audiences from one code base: **clients** raise and follow tickets in a company-scoped portal, **technicians** work tickets and log time, **managers** read productivity and client-workload analytics, and **administrators** configure PSA connections, field mappings, users, roles and reports. Every business record is scoped to an MSP organization, and that scope is applied centrally by the data layer, so a forgotten check in one screen or endpoint cannot widen it.

The build ran as ten delivery phases, all complete, followed by continuous hardening. The repository holds 264 commits, 30 database migrations and roughly 48 tables. Automated quality is 720 unit and integration tests plus a 10-spec Playwright browser suite, all green in CI, with secret scanning and dependency vulnerability scanning gating every merge.

The September 2026 work stream closed the operational gaps that separate "built" from "run": the sign-in service moved onto PostgreSQL so accounts survive a rebuild, a Content-Security-Policy is enforced, nightly encrypted backups run with a restore check, deploys no longer drop requests, the portal surfaces one "needs attention" list with a daily digest, and outbound mail can now leave through Microsoft 365 Graph rather than SMTP password sign-in.

What this document is for. It states what exists, what is outstanding and where the product should go next, with enough context for a reader who has not seen the code. Every figure is read from the live system or the repository on 28 September 2026, not estimated.

1.1 Status at a glance

Area	State
Product scope	Client portal, technician desk, manager analytics, administration — all shipped
Providers	Autotask (live), ConnectWise Manage (staging tenant); others are framework-ready, not shipped
Hosting	Single VPS, Docker Compose, host nginx with TLS; co-hosted with other TechPio sites
Identity	Keycloak 26, OIDC auth-code with PKCE; Microsoft Entra federation prepared, not enabled
Data protection	Per-tenant query filters, AES-256-GCM secret store, nightly restore-checked backups
Automated tests	720 unit/integration, 10 browser specs, CI gates on secrets and vulnerabilities
Deployment	Zero-downtime script; two consecutive production deploys with zero failed requests
Outstanding	Off-site backups, mail deliverability (SPF or Graph), first report schedules, ConnectWise closed-status mapping

1.2 Live production snapshot, 28 September 2026

Measure	Value
Tickets held	150
Ticket notes	362
Attachments	37
Time entries	16, totalling 65.5 hours
Staff users	41, in 7 departments, across 7 roles
Client companies	11
Client portal users	1
Field-mapping rules	52
Audit-log entries	269
Database size	11 MB across 48 tables, 30 migrations applied
Backups retained	12 nightly sets, 85 MB, each restore-checked
Container uptime	Database 3 weeks; application containers 5 to 6 days (last deploy)

The figures describe a system in early production use: the integrations and reporting are real, but adoption is thin. One client portal user and 16 time entries mean the client-facing and time-tracking halves of the product have not yet been rolled out to the people they were built for. That, rather than missing features, is the single biggest lever on value today.

2. What the product does

2.1 For clients

A client signs in and sees only their own company's tickets, enforced in the data layer rather than in the user interface. They can raise a ticket, follow the conversation, attach files, and read progress. Internal notes and files attached to internal notes never leave the staff side; this is covered by explicit tests, because a leak here would be a breach of the customer's trust in the MSP.

The client area (the Control Panel) also carries scheduled reports, business reviews, announcements, frequently-asked-questions content, and per-client branding. The reporting parts are live; the content parts (announcements, FAQ articles, per-client branding) are built but hold no data yet.

2.2 For technicians

The ticket desk is the daily surface: a filtered ticket list, a ticket page with the full conversation, and one composer that does three things in one action — write an internal note or a public reply, change the status, and log time. The composer opens on **Internal note**, and offers **Public reply** only when the ticket actually has a contact who would receive it, so nobody writes a reply into the void.

Notes and files carry the identity of the person who wrote them, not the integration account, and the thread distinguishes client messages from staff messages by colour and label. A deep link opens the same ticket in the underlying PSA, for the work the portal deliberately does not duplicate.

2.3 For managers

Managers get technician productivity over any calendar window (today, yesterday, this and last month, this and last quarter), a weighted productivity score with an operational-indicator guardrail, team comparison, trend, and CSV export by name. A client-workload view shows volume, resolution time, SLA attainment and who did the work, and every figure on the dashboard opens the tickets behind it rather than being a dead number.

2.4 For administrators

Administration covers PSA connections (credentials encrypted, never shown again), field mappings with versioning and audited rollback, users and roles with permission templates and per-user overrides, board access, the audit log, job monitoring with dead-letter reprocessing, integration health, email setup, and scheduled reporting. Staff can be imported from the PSA or from a spreadsheet, and a technician may exist in the portal without existing in the PSA.

3. Architecture and technology

Layer	Choice	Notes
Web	Next.js 15 App Router, React 19, TypeScript, Tailwind, React Query, Zod	Server-side proxy holds tokens; no access token ever reaches browser JavaScript
API	ASP.NET Core .NET 9	Permission-claim authorization, RFC-7807 errors, correlation IDs
Worker	.NET 9 background host	Sync polling, reports, digests, rollups, backfills
Data	PostgreSQL 17	EF Core with a global per-tenant query filter and write guards
Identity	Keycloak 26 on PostgreSQL	OIDC auth-code + PKCE; accounts survive container rebuilds
Secrets	AES-256-GCM encrypted rows	PSA credentials and mail secrets; plaintext never in the database or logs
Delivery	Docker Compose, GitHub Actions	Zero-downtime deploy script; host nginx terminates TLS

The repository is a monorepo: [apps/](#) holds the API, worker and web app; [packages/](#) holds domain, application, infrastructure, the provider-neutral [psa-core](#) contract and the per-provider connectors; [infrastructure/](#) holds Docker, nginx and operational scripts; [docs/](#) holds architecture, deployment, security, integration and QA documentation.

Two design decisions shape everything else. First, **the connector contract is provider-neutral**: a connector answers a capability matrix, and the portal adapts rather than assuming one vendor's behaviour. Second, **tenancy is enforced at the data layer**: the database context applies the organization filter to every query and stamps and guards it on every write, so a missing check in a controller cannot leak another MSP's data. An unresolved tenant scope matches nothing rather than everything, which is the safe direction to fail in.

3.1 Environments

Environment	Where	Purpose
Production	VPS 2.25.84.119, /opt/deskportal , Docker Compose	piomanage.com and auth.piomanage.com
Local development	Developer machine, SQLite "local mode"	Runs the whole product without Docker
CI	GitHub Actions	Build, tests, browser suite, secret and dependency scans

4. PSA integration and the sync engine

4.1 Capability matrix

Both shipped connectors are read-write. The differences matter operationally, so the portal asks the connector rather than assuming.

Capability	Autotask	ConnectWise Manage
Create and update tickets	Yes	Yes
Public and internal notes	Yes	Yes
Choose note email recipients	No (Autotask workflow decides)	No (no such field on the note)
Attachments, download, sweep	Yes, sweep supported	Yes, no tenant-wide sweep
Time entries	Yes	Yes
SLA data, custom fields	Yes	Yes
Inbound webhooks	Yes	Yes
Outbound webhooks	No	Yes
Maximum page size	500	1000
Maximum attachment size	6 MB	60 MB
Rate-limit model	Threshold per hour	Concurrent requests
Authentication	API key	Basic authentication

4.2 How synchronization works

The worker polls every connection every five minutes and pages through everything the provider has changed since the last successful run. Each ticket is upserted through the mapping engine, which translates provider values into portal values. Notes, attachments and time entries follow.

Four rules earned their place the hard way and are worth stating, because they are the difference between a sync that looks fine and one that is correct:

1 A discovered option carries two values. What the portal sends to the provider and what arriving tickets are labelled with are not the same string. Conflating them failed silently; the tell was a ticket whose PSA status and portal status were identical, meaning nothing was mapping.

1 Pagination must be real. All three connectors once declared "no more pages" after the first page, and both test doubles hid it, so only the first 100 tickets per run were imported.

1 A newly captured field must join the update hash, or existing rows never backfill when the meaning of "changed" widens.

1 History is dated when it happened. The daily rollup therefore backfills days older than its window, otherwise past events could never become facts.

Loop prevention is explicit: every write the portal makes is recorded as a sync event with an idempotency key and an echo marker, so the portal never re-imports its own change as if it were the provider's.

4.3 Field mapping

Mappings resolve across eight scopes, are versioned, and roll back as a set with an audit entry. The Mappings page shows whether the saved snapshot still matches the live rules and offers a one-click snapshot, because a rollback to a stale snapshot silently deletes rules that were added since. Production holds 52 rules across the two providers.

5. Reporting, analytics and email

5.1 What is produced

Report	Audience	Delivery
Technician productivity	Managers	On screen, CSV, scheduled PDF
Scheduled staff reports	Managers	Daily, weekly, monthly or quarterly at 07:00 organization time, PDF plus CSV by email
Client business review (QBR)	Clients	Quarter against previous quarter: SLA, resolution average and median, categories, priorities, who did the work, oldest open ticket. Scheduled or on demand
Client workload	Managers	Volume, hours, SLA, per-company drill-through
Activity rollup	Platform	Daily facts from raw events, with backfill
Needs-attention digest	Administrators	Daily at 07:30 organization time, only when something is wrong

Reports render as PDF with an embedded font, so output does not depend on fonts installed on the server, and every scheduled run is stored in the portal even when email delivery fails.

5.2 Email delivery

Outbound mail has two routes. **SMTP** works today through Microsoft 365 direct send on port 25 with no login. **Microsoft 365 through the Graph API** was added in September: the organization's own app registration signs in with a client secret, and mail leaves from a real mailbox, which is what makes it pass SPF and DKIM.

Deliverability is the open item. Direct send from this server is unlisted in the domain's SPF record, so mail lands in Junk. Either add `ip4:2.25.84.119` to the SPF record, or switch to the Graph route, which needs no DNS change. The Graph route is the better long-term answer.

6. Security and data protection

Control	Implementation
Tenant isolation	Global query filter on the database context plus write guards; adversarial cross-tenant tests
Client scoping	Client users see only their own company's tickets, and only public notes
Authentication	Keycloak OIDC, authorization-code flow with PKCE; tokens in httpOnly cookies, never in browser JavaScript
Authorization	Permission-claim policies on every endpoint, with a test that walks every controller and asserts the refusal
Secrets	AES-256-GCM encrypted rows; PSA credentials and mail passwords are write-only from the interface
Outbound request safety	Egress guard blocks private and reserved addresses, so an administrator's form cannot reach internal services
Browser policy	Content-Security-Policy enforced in production, with violation reporting; HSTS on the public hosts
Attachments	Validated, scanned, quarantined on detection, stored under randomized keys, served by time-limited signed URLs, downloads audited
Audit	Append-only audit log over configuration changes, mapping snapshots, user and role changes, email setup and digests
Backups	Nightly, encrypted off-site copy available, restore-checked every run

Known security gaps, all environmental rather than code defects: no live dynamic scan or penetration test has been run against the deployed stack, no load test has been executed against production-like data, and no disaster-recovery restore drill has been performed end to end on the real host.

7. Operations

7.1 Deployment

Deployment is a single script on the VPS. The API starts a second container from the new image beside the old one and takes traffic only once its readiness endpoint answers; the old container then stops gracefully. The web app starts a standby container on a second local port, nginx is pointed at it with a graceful reload, the main container is recreated, and nginx is pointed back. The worker is recreated in place because it serves no requests.

Deploy	Probe requests during the swap	Failed
22 September, first run	494	0
22 September, second run	430	0

Before this, a deploy replaced containers in place and every click during the gap returned an error; that happened five times in September while the owner was testing. The failure is now designed out rather than scheduled around. Migrations must stay additive, because the new API runs its migrations while the old one is still serving.

7.2 Backups

A nightly job at 03:45 dumps the application database, tars the attachments volume and dumps the sign-in database. Every dump is restored into a scratch database before it is counted as good; the most recent run restored 48 tables and 150 tickets. Retention is 14 days, currently 12 sets and 85 MB.

An encrypted off-site copy is built and drilled: files are encrypted on the server with a passphrase the owner holds, uploaded to any S3-compatible bucket, and verified by size. It switches on when the settings file exists on the server, which is the one outstanding step.

7.3 Monitoring and health

Integration Health shows connection status, last sync, pending and dead-letter jobs, failed sync events, storage usage and email status. On top of it sits the **needs-attention list**, which states in one place what is quietly wrong: failed or stalled connections, tickets that never reached the PSA, closed tickets with no closed date, reports that were not emailed, and missing mail setup once a schedule depends on it. The same list is emailed daily when it is not empty.

Today that list holds exactly one item: six ConnectWise tickets whose status is "Completed" but which carry no closed date, so resolution-time and SLA reports leave them out.

8. Quality assurance

Layer	Coverage
Unit and integration	720 tests: tenant isolation, RBAC, connectors and their certification suite, mapping engine, sync rules, resilience, reporting, email, attachments, authorization over every endpoint
Browser	10 Playwright specs over the real application: dashboard, reports, email settings, public site, run in CI with the production security policy enforced
Contract	A connector certification suite every provider implementation must pass
Load	k6 scripts authored; not yet run against production-like data
Manual	A 239-case feature audit in September merged test coverage, production evidence and a page-by-page walkthrough

The September audit found 63 features working with production evidence, 53 proven by tests, 57 partly proven, 35 unverified, 29 built but not in use, and 2 defective. All five code defects it found were fixed and deployed, and every improvement item it raised is now closed.

The 29 "built but not in use" features are the most commercially interesting line in that audit: the product already contains machinery nobody has switched on. Section 10 turns that into a plan.

9. What is outstanding

9.1 Waiting on the owner

Item	Why it matters	Effort
Off-site backup settings file	The VPS is currently the only copy of everything. A lost host loses the portal	15 minutes
ConnectWise closed statuses	Six tickets are excluded from resolution and SLA reporting. Either set the board status's Closed flag, or map "Completed" to Resolved	10 minutes
Email deliverability	Add the server to the domain SPF record, or complete the Microsoft 365 app registration for the Graph route	30 minutes
First report schedules	Scheduled reporting is built and idle until someone names recipients and a cadence	15 minutes
Needs-attention digest recipients	The daily digest is off until addresses are entered	2 minutes
ConnectWise tenant decision	The connection points at a staging tenant; production data needs the production tenant	Decision
Microsoft Entra sign-in	Tenant and client IDs are needed to federate staff sign-in instead of issuing passwords	Decision plus 30 minutes
Technician to PSA linking	Only 3 of 41 staff are linked to a PSA identity, which limits per-technician reporting	20 minutes
AI assistant note policy	The assistant currently reads internal notes; confirm that is intended before it is used with clients	Decision

9.2 Engineering gaps

Gap	Impact	Recommended response
No live dynamic scan or penetration test	Unknown runtime exposure	Commission one before selling to a security-conscious client
No load test against production-like data	Performance behaviour under real volume is unproven	Generate volume and run the existing k6 scripts
No disaster-recovery drill on the real host	Restore steps are documented but never rehearsed end to end	Do one timed drill and record the result
Polling every five minutes	Ticket changes appear with up to five minutes of delay	Accept for now; move to webhooks when near-real-time is needed
Client adoption is one user	The client-facing half of the product is unexercised	Onboard one friendly client as a pilot
Cross-browser validation	Only Chromium is exercised automatically	Add Firefox and WebKit to the browser suite

10. Recommended enhancements

The recommendations below are ordered by return on effort, not by novelty. The first tier turns existing machinery on; the second tier adds what an MSP buyer expects to see in a portal; the third tier is platform work that only pays off once the product is sold to more than one MSP.

10.1 Tier one — switch on what is already built (days, not weeks)

- 1 **Run a client pilot.** One client company, three named users, their real tickets. Everything needed for this exists. This converts a technically finished product into evidence that it works, and will produce a better feature list than any internal review.
- 1 **Turn on announcements, FAQ articles and per-client branding.** The tables, the administration screens and the client views exist and hold no data. A branded portal with the client's logo and a short FAQ is what makes the portal feel like the MSP's own product rather than a ticket list.
- 1 **Fill in business hours, holidays and escalation levels.** These drive honest SLA measurement. Without them, SLA attainment is measured against wall-clock time, which flatters or punishes the team unfairly.
- 1 **Link the remaining technicians to their PSA identities.** Per-technician productivity is only as complete as this mapping, and 3 of 41 are linked today.
- 1 **Publish the first scheduled reports.** A weekly technician report to the manager and a quarterly business review to each client cost minutes to configure and are the most visible proof of value the product can produce on its own.

10.2 Tier two — the features an MSP buyer expects next

- 1 **Near-real-time updates through webhooks.** ConnectWise supports outbound webhooks and both providers accept inbound ones. The receiving endpoint with signature and timestamp validation is already built and live. What is missing is the other half: the job that consumes a received event currently acknowledges it without applying it, and nobody subscribes the portal to the provider's events. Finishing both replaces five-minute polling with event-driven updates and removes the most noticeable difference between the portal and the PSA's own interface. **Effort: medium. Impact: high.**
- 1 **Customer satisfaction (CSAT) capture.** One question when a ticket closes, stored against the ticket and the technician, reported per client and per period. It is a small feature that gives the MSP something to sell with, and the reporting pipeline to carry it already exists. **Effort: small. Impact: high.**
- 1 **SLA engine with breach warnings.** SLA data arrives from both providers, and the needs-attention list is the natural place to surface "three tickets will breach within two hours". This turns a reporting product into an operational one. **Effort: medium. Impact: high.**
- 1 **Approval workflows.** The approvers table exists and is unused. Change approvals, quote approvals and after-hours authorization are the requests MSP clients actually make of a portal. **Effort: medium. Impact: medium to high.**
- 1 **Assets and devices in the client portal.** Both connectors report assets, and a devices table exists. Showing a client their own estate, with the tickets raised against each device, is a differentiator few PSA portals do well. **Effort: medium. Impact: medium.**
- 1 **A knowledge base with client-visible and staff-only articles.** The FAQ table is the seed. Tied

to ticket deflection reporting, it is the clearest path to reducing ticket volume, which is what an MSP's own margin depends on. **Effort: medium. Impact: medium.**

1 **Mobile-first technician view or progressive web app.** The interface is already responsive and verified at phone width; an installable application with push for assignment and escalation would make the portal usable on site. **Effort: medium. Impact: medium.**

10.3 Tier three — platform and commercial readiness

1 **Second MSP tenant onboarding.** The data model is multi-tenant throughout, but the product has only ever run one organization. Onboarding a second one — self-service organization creation, per-tenant branding, per-tenant mail — is what converts an internal tool into a SaaS product. **Effort: large. Impact: high, and it is the whole commercial thesis.**

1 **Subscription and usage metering.** Seats, tickets and storage per organization, with invoicing. Nothing bills today. **Effort: large. Impact: high once tenant two exists.**

1 **Provider wave two: HaloPSA, Syncro, ServiceNow.** The connector contract and its certification suite were designed for exactly this, and the marketing pages already describe those providers honestly as planned. Each new provider widens the addressable market without touching the core. **Effort: medium per provider. Impact: high commercially.**

1 **Observability: OpenTelemetry traces, metrics and uptime alerting.** Today the evidence is structured logs and the health page. An external uptime probe and alerting on sync failure would mean nobody has to notice a problem by hand. **Effort: small to medium. Impact: medium.**

1 **AI assistance where it is defensible.** Draft-reply suggestions from ticket history, automatic ticket summarization for handover, and category suggestion at intake. The assistant framework and its settings already exist. Keep it drafting for a human rather than acting, and keep internal notes out of anything client-facing. **Effort: medium. Impact: medium to high.**

1 **PostgreSQL row-level security as a second line.** Tenant isolation today rests entirely on the application's query filters and write guards, which are well tested but are one layer. Enabling row-level security on tenant tables, with the organization set per connection, means the database itself refuses a cross-tenant read even if application code is wrong. This is the single highest value security change available, and it is the control an enterprise security review asks about. **Effort: medium. Impact: high.**

1 **Data residency and export.** Per-tenant export of tickets, notes and attachments, plus a documented deletion path, is the kind of question that appears in the first enterprise security review the product faces. **Effort: medium. Impact: medium.**

10.4 If only three things are done next

1 Off-site backups and the ConnectWise closed-status fix, because they are cheap and they protect what already exists.

1 A client pilot with branding, announcements and a scheduled business review, because it converts the product into evidence.

1 Webhook-driven updates and CSAT, because together they answer the two questions a buyer asks first: is it live, and does it prove my team is good.

11. Risk register

Risk	Likelihood	Impact	Mitigation in place	Gap
Loss of the VPS	Low	Severe	Nightly restore-checked backups on the host	No off-site copy until the settings file is created
Mail lands in Junk	Certain today	Medium	Graph route built; SPF change identified	Neither route completed
Credentials exposure	Low	Severe	Encrypted secret store, write-only fields, audit	No penetration test yet
Provider API change	Medium	Medium	Capability matrix, connector tests, health page	No alerting when a connection degrades outside the digest
Silent reporting error	Medium	Medium	Needs-attention list catches dateless closures and undelivered reports	Depends on someone reading it, until digest recipients are set
Single-operator knowledge	Medium	Medium	Documentation in the repository, runbooks for deploy and restore	No second person has rehearsed a restore

Appendix A. Screens and routes

A.1 Public site

Route	What it is for
/	Product home: one client portal for the PSA the MSP already runs
/platform	How the platform works
/features, /features/[slug]	Feature catalogue and per-feature detail
/integrations, /integrations/[platform]	Supported PSA platforms; per-platform page that says plainly whether a connector ships today
/about, /faq, /contact, /book	Company, questions, enquiry form, meeting request
/security, /privacy, /terms	Security posture, privacy policy including enquiry retention, terms
/login	Sign-in entry point

A.2 Staff dashboard

Route	What the user can do
/dashboard	Overview: the day's figures, each one opening the tickets behind it
/dashboard/tickets	Filterable ticket list
/dashboard/tickets/new	Raise a ticket
/dashboard/tickets/[id]	Ticket detail: conversation, attachments, status, assignment, time timer
/dashboard/analytics	Productivity dashboard: SLA attainment, open, created, resolved, top performer
/dashboard/analytics/technicians	Per-technician hours and output over any window
/dashboard/analytics/clients	Client workload with drill-through
/dashboard/analytics/coverage	Portal coverage
/dashboard/reports	Scheduled staff reports and run downloads
/dashboard/connections	PSA connections: add, test, sync, settings, logo, field refresh
/dashboard/mappings	Field mapping rules, versions, snapshot status, rollback
/dashboard/health	Integration health, needs-attention list, email setup, unsynced tickets
/dashboard/assistant	AI assistant settings and use
/dashboard/users, /dashboard/users/[id]	Users and access: list, create, import, bulk actions; per-user roles, departments, teams, board access, templates
/dashboard/roles, /dashboard/permissions	Roles and permissions; who effectively holds what
/dashboard/departments	Departments and teams
/dashboard/jobs	Background jobs, with reprocessing
/dashboard/audit	Audit log
/dashboard/enquiries	Inbound public enquiries with triage status

Route	What the user can do
/dashboard/notifications, /dashboard/profile	Notifications and own profile

A.3 Client control panel

Route	What the client can do
/control-panel/instructions	Maintain ticket instructions for their users
/control-panel/users	Invite and manage their own users and access
/control-panel/approvers	Maintain the approver list
/control-panel/escalation	Maintain the escalation path
/control-panel/business-hours, /control-panel/holidays	Business hours, holidays, import holidays from the PSA
/control-panel/announcements, /control-panel/knowledge-base	Announcements and FAQ articles
/control-panel/accounts	Agreements, monitored queues, devices
/control-panel/reports	Tickets by status, scheduled reports, run history
/control-panel/branding	Display name, logo, accent colour
/control-panel/notifications	Notification history

A.4 Web application routes that are not pages

Route	Purpose
/api/auth/login, /callback, /logout, /session	OIDC authorization-code with PKCE; tokens land in httpOnly cookies
/api/bff/[...path]	Server-side proxy to the API: attaches the token, refreshes on expiry, retries a request that never reached the API
/api/attachments/blob	Serves attachment bytes for a signed download URL
/api/csp-report	Receives browser security-policy violation reports
/version	Lets an open tab notice it is running a stale bundle

Appendix B. API surface

B.1 Tickets

Endpoint	Purpose	
GET /api/tickets, GET /api/tickets/{id}	List and detail, scoped by permission and company	
POST /api/tickets	Create	
POST /api/tickets/{id}/comments	Add a note, internal or public	
GET /api/tickets/{id}/recipients	Who a public reply would reach	
POST /api/tickets/{id}/contact/refresh	Re-read the contact from the PSA	
POST /api/tickets/{id}/status	Change status	
GET /api/tickets/{id}/assignees, PUT /api/tickets/{id}/assignment	Assignment	
GET	POST /api/tickets/{id}/time, PUT	DELETE .../
POST /api/tickets/{id}/attachments, GET .../download, GET /api/attachments/blob	Upload, mint a signed URL, serve bytes	

B.2 Dashboards and reporting

Endpoint	Purpose	
GET /api/dashboard/technician	daily	clients
GET /api/dashboard/team/export	Team export	
GET	PUT /api/reports/settings	Organization
GET	POST	DELETE /ap
GET /api/reports/runs, GET /api/reports/runs/{id}/pdf	csv	Run history
GET /api/reports/qbr.pdf	Client business review for a company, year and quarter	
GET /api/reports/technician-productivity.pdf	Technician productivity PDF	

B.3 Administration

Group	Endpoints
Connections	list, create, update, enable, logo, test, time-entry check, sync, field metadata, settings

Group	Endpoints
Mappings	list, upsert, delete, version history, save snapshot, snapshot status, rollback
Jobs	list by status, reprocess
Sync repair	GET /api/admin/tickets/unsynced, POST /api/admin/tickets/{id}/resync
Health	GET /api/admin/health, GET /api/admin/attention, PUT /api/admin/attention/digest, POST /api/admin/attention/digest/send, GET /api/admin/storage
Audit	GET /api/admin/audit
Users	list, detail, create, import, update, activate, delete, bulk actions, photo
Roles and permissions	roles CRUD, catalogue, effective permissions, templates, apply template
Org structure	departments and teams CRUD, membership, board access and grants
PSA identities	list provider technicians, provision, read and set a user's PSA identity
Email	status, settings read, write and delete, send test

B.4 Client, identity and integration

Endpoint	Purpose	
GET /api/control-panel/capabilities	Which sections this client may use	
Control panel groups	instructions, users, approvers, escalation, holidays, devices, business hours, PSA import and view	
Client content	announcements, branding, FAQ, reports, schedules, runs and downloads	
GET /api/me	Current user, tenant and permission claims	
GET /api/notifications, /history, `GET	PUT /api/profile`	Notifications
GET /api/organizations/connections	Connections visible to the caller	
GET /api/assistant/availability, POST /api/assistant/tickets/{id}, `GET	PUT /api/assistant/settings`	AI assistant
POST /api/public/enquiries/contact, /meeting	Public enquiry and meeting request	
GET /api/admin/enquiries, POST /api/admin/enquiries/{id}/status	Enquiry triage	
POST /api/webhooks/{connectionId}	Inbound provider events, signature and timestamp validated	

Appendix C. Permissions and roles

C.1 Permission keys

Authorization is evaluated against permission claims, never role names. The catalogue holds 27 keys.

Group	Keys
Platform	<code>platform.organizations.manage</code> , <code>platform.health.view</code> , <code>platform.settings.manage</code>
Organization and integrations	<code>org.manage</code> , <code>connections.manage</code> , <code>connections.view</code> , <code>mappings.manage</code> , <code>mappings.view</code>
Users and roles	<code>users.manage</code> , <code>roles.manage</code> , <code>clientusers.manage</code>
Tickets	<code>tickets.view.all</code> , <code>tickets.view.assigned</code> , <code>tickets.view.company</code> , <code>tickets.view.own</code> , <code>tickets.create</code> , <code>tickets.note.public.add</code> , <code>tickets.time.log</code> , <code>tickets.update</code>
Reporting	<code>reports.view</code> , <code>productivity.team.view</code> , <code>productivity.own.view</code>
Operations and audit	<code>integration.health.view</code> , <code>jobs.manage</code> , <code>audit.view</code> , <code>security.config.view</code>
Enquiries	<code>enquiries.view</code>

Each key carries a scope: All, Department, Team, Assigned, Own, Selected or None. A per-user override replaces the role-derived scope rather than merging with it, which keeps "why can this person see this" answerable.

C.2 Built-in roles

Role	In one line
Platform super administrator	Every key at its declared scope
MSP administrator	Everything within one MSP organization
Manager	Read connections and mappings, all tickets, log time, update, reporting and team productivity, health, enquiries
Technician	Assigned tickets, public notes, time logging, updates, own productivity
Client administrator	Their company's tickets, create, public notes, manage their own users, reporting
Client user	Their own tickets, create, public notes
Auditor	Audit log, security configuration, integration health

C.3 Permission templates

Eight templates ship, each expressed as the difference from a base role: Full Administrator, Service Desk Manager, Senior Technician, Standard Technician, Dispatcher, Billing User, Auditor, Read Only. Storing only the difference means a change to a base role still reaches everyone who inherits it.

Appendix D. Background processing

Service	What it does	Cadence
Polling sync	Full inbound sync for every enabled connection, through the same code path as the manual Sync button	Every 5 minutes
Background job poller	Claims due jobs across tenants and runs them with retry, backoff and dead-letter	Every 10 seconds
Activity rollup	Recomputes daily facts over a rolling seven-day window and bounds the raw event log	Hourly
Client scheduled reports	Generates and delivers client report schedules that fall due	Every 5 minutes
Staff scheduled reports	The MSP's own daily, weekly, monthly and quarterly reports at 07:00 organization time	Checked every 5 minutes
Needs-attention digest	Emails the attention list once a day at 07:30 organization time, only when it is not empty	Checked every 10 minutes
Enquiry retention	Deletes public enquiries past the published retention period, sweeping by age so a missed day self-repairs	Daily
Author backfill	Fills author identities on notes and attachments imported before identities were retained	Once per start
Contact backfill	Fills the customer contact on tickets imported before contacts were retained	Once per start

Appendix E. Where everything lives

E.1 Code and deployment

Thing	Location
Source repository, local	E:\autotask\DPI-Autotask\desk-portal on the development machine
Source repository, remote	github.com/Dalbeirdev/deskportalpsa, branch main
Production host	VPS at 2.25.84.119, application under /opt/deskportal
Production URLs	piomanage.com (portal) and auth.piomanage.com (sign-in)
Compose files	infrastructure/docker/docker-compose.prod.yml plus docker-compose.hostproxy.yml
Environment file	infrastructure/docker/.env.prod on the server, never in the repository
Deploy command	/opt/deskportal/infrastructure/scripts/deploy-vps.sh
Backup command	/usr/local/sbin/deskportal-backup, nightly at 03:45, output in /var/log/deskportal-backup.log
Off-site settings	/etc/deskportal/offsite.env, template at infrastructure/scripts/offsite.env.example
Web server	Host nginx, site file piomanage, logs /var/log/nginx/piomanage.access.log and piomanage.error.log
Backups on disk	/var/backups/deskportal, 14-day retention

E.2 Documentation in the repository

Path	Contents
docs/architecture/	The ten-phase delivery plan and phase-by-phase acceptance evidence
docs/deployment/	Local run, go-live for piomanage, release checklist, backup and recovery
docs/integrations/	Autotask and ConnectWise integration guides
docs/security/	Security posture and an OWASP-mapped security review
docs/testing/	Final QA report with the test-case matrix and known limitations
docs/qa/generator/	The generator behind the QA test plan and tester workbook
CHANGELOG.md	Release history

E.3 Running it on a development machine

The product runs locally without Docker in "local mode", which uses a SQLite database and a local file store. Two run configurations start it: the API on port 5400 and the web application on port 3400; a third starts the web application with the production security policy enforced, which is what the browser test suite uses.

Appendix F. Delivery history and decisions

F.1 The ten phases

Phase	Name	Outcome
1	Discovery and architecture	Approved plan, connector-first design
2	Foundation	Monorepo, multi-tenant schema with database-level isolation, Keycloak sign-in, permission-claim authorization, encrypted secret store, structured logging, CI
3	Integration framework	Provider-neutral connector contract, capability model, certification suite, eight-scope mapping engine, retry and circuit breaker, loop prevention, webhook ingress, job retry and dead-letter
4	Autotask	REST connector and ticket sync engine
5	ConnectWise Manage	REST connector and cross-provider normalization parity
6	Client portal	Company-scoped tickets, public notes only, notifications, profile
7	Technician and manager dashboards	Weighted productivity score, metrics, team comparison, trend, export
8	Administration	Connections, mapping versioning with audited rollback, users and roles, audit log, health, job monitor
9	Security and performance	Egress guard, adversarial cross-tenant tests, performance indexes, load scripts, backup runbook, OWASP-mapped review
10	Final QA and readiness	Accessibility and contrast fixes, mobile navigation, reduced-motion support, QA report

F.2 What was built after the phases, August to September 2026

Theme	Work
Correctness of the sync	Date, status, priority, queue and category mapping corrected at the root; pagination fixed, which had silently capped imports at 100 tickets per run; the rollup taught to backfill history
Attribution	Notes, attachments and time attributed to the person who did the work rather than the integration account; backfills for records imported before identities were retained
The ticket page	One composer for note, status and time; internal-first with public reply only when a contact exists; deep link into the PSA; image attachments shown in the thread
Reporting	Calendar periods and CSV by name, SMTP delivery, scheduled staff reports as PDF and CSV, client business reviews, email setup in the portal
Client-facing site	Honest per-platform integration pages, legal pages completed, enquiry retention published and enforced
Security	Content-Security-Policy first reported then enforced, HSTS, endpoint authorization tests over every controller, attachment rules that keep internal files internal
Operations	Keycloak moved to PostgreSQL, nightly restore-checked backups, log rotation, per-site web logs, zero-downtime deploys, needs-attention list and daily digest, encrypted off-site backup

Theme	Work
Mail	Sign-in only where the server offers it, then Microsoft 365 through the Graph API as the recommended route

F.3 Decisions worth remembering

- **The PSA stays the system of record.** The portal never becomes the place where truth lives, which is what makes it safe to adopt beside an existing service desk.
- **Tenancy is enforced in the database.** A missing check in a controller cannot leak another organization's data.
- **Capabilities are asked, not assumed.** Each connector answers what it supports, so the interface can hide a control the provider would ignore rather than offering a promise it cannot keep.
- **Polling over webhooks, for now.** Five-minute polling is simple and reliable; event-driven updates were left as a later upgrade rather than a half-built dependency.
- **Secrets are write-only from the interface.** Credentials can be replaced but never read back, including by an administrator.
- **Failure is reported, not hidden.** Unmapped provider values, undelivered reports, tickets that never reached the PSA and stalled connections all surface in one list rather than in a log nobody reads.

Appendix G. Infrastructure

G.1 Production services

The production stack is one Docker Compose project. No application container publishes a public port; the host's own nginx terminates TLS and proxies to the web container.

Service	Image or build	Purpose
postgres	postgres:17-alpine	The application database and Keycloak's own database, with a readiness check
keycloak	quay.io/keycloak/keycloak:26.0	Sign-in and token issuance, persisting to PostgreSQL so accounts survive a rebuild
api	Built from the repository	The .NET API. Runs migrations on start and blocks outbound requests to private addresses
worker	Built from the repository	Sync, jobs, reports, digests and backfills
web	Built from the repository	The Next.js application, which reaches the API over the internal network
cloudflared	cloudflare/cloudflared	Optional: publish with no open host ports through a Cloudflare tunnel
caddy	caddy:2-alpine	Optional: terminate TLS itself when the host has no web server

Three exposure routes exist and are mutually exclusive: the host's own nginx (in use), Caddy at the edge, or a Cloudflare tunnel. Every service rotates its logs at 20 MB with five files kept. Secrets and bootstrap values come from an environment file on the server that is deliberately not in the repository; the stack refuses to start without them.

G.2 Operational scripts

Script	Purpose
deploy-vps.sh	Zero-downtime deploy: second API container, standby web container, nginx cut-over, worker recreated in place
backup-vps.sh	Nightly database, attachments and sign-in backups with a restore check and optional encrypted off-site upload
offsite.env.example	Template for the off-site settings file, documenting Backblaze B2, Wasabi, AWS S3 and Cloudflare R2
backup.sh	Generic backup for a host with the database and object-store tools installed

Supporting configuration: the nginx site template with per-host HSTS, the Keycloak realm definition, a script that federates sign-in to Microsoft Entra, and a script that provisions technicians.

Appendix H. Continuous integration

Every push and pull request to the main branches runs five jobs. A failure in any of them blocks the merge.

Job	What it does
Backend	Restores and builds the solution with warnings treated as errors, then runs the unit and integration suite and publishes the results
Web	Installs dependencies, lints, type-checks and builds the Next.js application
Browser tests	Installs Chromium, builds the API, and runs the Playwright suite against the real application with the production security policy enforced, so a policy change that breaks a page is caught here
Secret scan	Scans the full history with gitleaks and fails on any finding
Dependency scan	Lists vulnerable packages including transitive ones and fails if any are found

The secret scanner's configuration extends the default rules and allow-lists only development fixtures and one test key, so real credentials cannot be excluded by accident.

Appendix I. Test inventory

I.1 Automated suites

Suite	Size	Scope
Unit and integration	74 test classes, 720 tests	The whole backend, from tenancy to reporting
Browser	3 specs, 10 tests	The real application in Chromium, with the production security policy enforced
Connector certification	A shared suite each provider must pass	Autotask, ConnectWise and the mock provider, against recorded provider behaviour
Load	One k6 smoke scenario	Ramps to 25 concurrent users with thresholds per endpoint; needs a live stack, not yet run

I.2 What the backend tests cover

Theme	Classes
Security, tenancy, authorization	10, including isolation, audit and role-based access, an endpoint-by-endpoint authorization test, an effective-permission test and a golden permission matrix
Secrets and cryptography	2
Connectors and certification	8, including cross-provider normalization
Sync, jobs and resilience	7, including sync failure handling and unsynced-ticket repair
Attachments	5, plus 2 for connection logos
Tickets, notes and time entries	9, including ticket scope translated to SQL, reply recipients and time-entry failure
Client portal	6
Administration and users	9, including staff import and contact backfill
Analytics and productivity	7, including the needs-attention rules
Email and notifications	5, including Microsoft 365 Graph sending
Assistant, mapping, enquiries and provisioning	6

I.3 What the browser tests prove

- The overview opens with its navigation and the signed-in user.
- Technician hours offers calendar periods and states the range it is showing.
- A client portal page tells a staff account plainly that it is not for them.
- The public site loads, lists every platform, and neither promises a connector that does not exist nor hides the one that does.
- A report schedule can be created, refuses a bad address, runs, and produces a PDF.
- The page says when email is not set up and points at where to set it up.
- An administrator can add a mail account, and the password never comes back out.

Appendix J. Data model and change history

J.1 Entity groups

Group	Entities
Tenancy	MSP organization, PSA connection, client company, client user, organization email settings
Tickets	Ticket, note, attachment, time entry, with time-entry source and sync status
Identity and access	Application user, role, role permission, user role, PSA identity, permission override, permission template and entries, board access and grants
Organization structure	Department, team, and their memberships
Mapping	Field mapping and field mapping version
Sync	Sync event and background job
Control panel	Approver, escalation level, holiday, device, business hours, client access grant, ticket instruction, announcement, FAQ article, client branding, report schedule and run
Reporting	Staff report schedule and run
Analytics	Activity event and activity daily fact
Other	Assistant settings, audit log entry, marketing enquiry, encrypted secret blob

J.2 Schema history

Thirty migrations, from the initial schema in July 2026 to the most recent in September 2026. The sequence reads as the product's history: performance indexes, then the client control panel and its content, then time-entry provenance, enquiries, the encrypted secret store, the organization and authorization model, note and attachment authorship, activity events and daily facts, portal work attribution, staff reports, organization email settings, the attention digest and Microsoft 365 Graph email.

J.3 Security mechanisms and where they live

Mechanism	Where
Tenant query filters, write guards, fail-closed scope	The database context
Append-only audit log	The database context refuses to modify or delete an audit entry
Encrypted secret store	AES-256-GCM with a key supplied by configuration; production refuses to start with a development store
Outbound request guard	A message handler on connector clients that blocks private, loopback, link-local and cloud metadata addresses
Content-Security-Policy, framing and referrer headers	The web application's configuration, with violation reporting
Rate limiting, request size cap, CORS allowlist, problem details, correlation ids	The API startup and middleware
Attachment pipeline	Validate, scan, quarantine without storing infected bytes, randomized keys, signed time-limited URLs

Mechanism	Where
Sign-in	Authorization code with PKCE, tokens in httpOnly cookies, never in browser JavaScript

One layer, not two. Tenant isolation is enforced by the application's data layer. PostgreSQL row-level security is not in use. The filters are well tested, but adding row-level security would make the database refuse a cross-tenant read even if application code were wrong. Section 10.3 lists this as the highest-value security change available.